

中国研究生操作系统开源创新大赛

项目说明书

作品名称:	CPN-HRL-DAG
参赛队伍:	云边智链
完成日期:	2026 年 10 月

目 录

第 1 章 绪论.....	1
1.1 目的与意义.....	1
1.2 项目背景.....	2
第 2 章 技术理论.....	5
2.1 设计思想.....	5
2.1.1 问题建模与统一调度语义.....	5
2.1.2 分层动作与合法性约束.....	6
2.1.3 模块解耦与同场景评估.....	7
2.2 技术路线.....	7
2.2.1 两级策略网络与 PPO 更新.....	7
2.2.2 启发式残差与训练阶段.....	9
2.2.3 HRL 搜索和候选验证选择.....	10
2.3 代码原创说明.....	11
第 3 章 项目介绍.....	14
3.1 功能介绍.....	14
3.1.1 场景与资源配置.....	14
3.1.2 模拟器和策略接入.....	16
3.1.3 训练评测与结果追溯.....	17
3.1.4 运行部署与交付.....	18
3.2 软件界面.....	19
3.2.1 场景选择与方案对照.....	19
3.2.2 调度查看与结果导出.....	20
第 4 章 实验测试.....	22
4.1 测试说明.....	22
4.1.1 功能与约束测试.....	22
4.1.2 数据划分和性能协议.....	23
4.1.3 系统环境与复现方法.....	24
4.2 测试结果.....	25
4.2.1 功能完整性和平台验收.....	25
4.2.2 总体性能与计算代价.....	26

- 4.2.3 规模与资源分组.....27
 - 4.2.4 学习过程和组件贡献.....28
 - 4.2.5 调度案例与局限.....28
- 第 5 章 实现难点说明..... 30
 - 5.1 约束与调度语义一致性.....30
 - 5.2 分层学习稳定性与贡献归因.....31
 - 5.3 搜索计算开销与结果保护.....32
 - 5.4 平台与可复现交付..... 33
- 第 6 章 总结..... 35
 - 6.1 已完成工作与实验结论.....35
 - 6.2 现有限制与后续工作.....36
- 参考..... 37

第 1 章 绪论

CPN-HRL-DAG 面向云边端异构资源上的离线依赖任务调度，建立了统一的场景建模、合法动作约束、分层策略训练和可复现评估框架。学习模块分别决定任务选择和资源分配。系统还提供独立搜索与 HEFT 候选，在统一模拟器中验证完整调度，并选择完成时间最短的合法方案。本说明书分别讨论框架功能、学习表现和组合效果。

正式实验采用固定的 108 个验证场景、54 个基础 DAG 和三个随机种子。组合方法的平均归一化 makespan 为 0.918223，种子间样本标准差为 0.000498，归档调度合法率为 100%。

1.1 目的与意义

有前后依赖的计算任务需要按依赖关系执行。后继任务必须等待前驱输出，而各节点的计算速度、设备能力和通信条件会影响等待时间与执行时间。任务选择和节点分配需要一起考虑：计算更快的节点可能带来额外传输，局部较早完成的任务也可能占用后续关键任务所需的资源。

以一个抽象工作流为例，任务 A 的输出分别交给 B 和 C，任务 D 等待两者的结果后执行。B、C 可以在不同节点上处理。D 何时开始，取决于两条路径的结果何时到达其执行节点。调度需要在满足依赖和资源约束的前提下，降低全部任务的最晚完成时刻，即 makespan。每一步选择最快的设备，并不一定能达到这一目标。

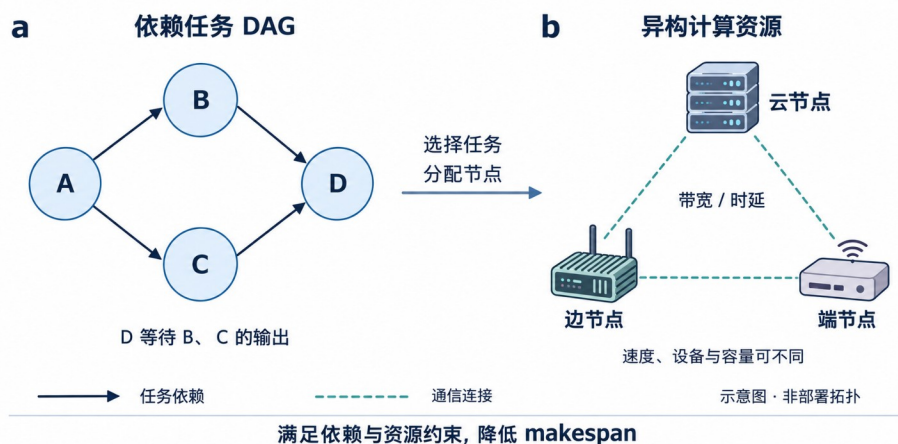


图 1-1 展示调度需求。左侧实线箭头表示任务依赖，中间蓝色箭头表示任务选择与节点分配的决策流程，右侧虚线表示资源间的通信连接。图中的节点数量和连线仅用于示意。实际场景未必同时包含三类节点。

赛题要求实现可训练的深度强化学习方法，也强调环境和资源可配置、策略可插拔，以及在同一组验证场景中公平比较。项目既需要能够实现调度决策，也要可以提供可复用的实验平台。为此，数据适配与策略相互独立，合法性由统一环境约束，结果可以追溯到对应的场景、配置和权重。

表 1-1 赛题要求与项目能力对应

赛题要求	项目实施与证据	完成范围
可训练的 DRL 调度框架	两级 HRL 与比较模型训练入口；正式三种子训练记录	已完成既定批次；稳定学习增益仍待改进
环境和资源配置可替换	Scenario、执行通信模型、资源生成配置与固定划分	支持配置替换，不等于全部配置均经正式实验
统一策略接口与公平比较	SchedulerPolicy 和同场景 Evaluator；HEFT 及学习策略共用模拟器	比较采用统一依赖、设备和插入时间线语义
合法动作与结果文件	ready/node mask、环境检查、候选重放及 CSV/JSON 产物	有显式处理及对应测试；详见第 4、5 章
国内开源操作系统支持	openEuler 与 openKylin 虚拟机短训练、验证和运行记录	两系统均有 CPU 运行结果；具体证据与范围见第 4 章

完成时间受任务顺序和节点分配共同影响，需要在同一模型中比较启发式、学习策略和搜索各自的作用。可复用框架可以减少比较中的语义差异，让性能变化更容易解释和复查。一个较低的平均值本身还不足以说明这些变化。

1.2 项目背景

Topcuoglu 等（2002）提出的 HEFT 根据任务优先级和异构节点上的预计完成时间进行列表调度，是本项目采用的确定性参考方法。它的规则清晰，适合作为统一比较基准。固定排序和局部放置也有局限，难以覆盖任务顺序、通信和资源竞争的所有组合。本项目将 HEFT 用作基线、学习先验和完整候选。

学习方法尝试从场景和调度状态中学习决策偏好。Schulman 等（2017）提出的 PPO 用于策略更新，Hochreiter 和 Schmidhuber（1997）提出的 LSTM 与 Veličković 等（2018）提出的 GAT 分别用于表示序列和图关系。Yue 等（2026）提出的 CPN-HRL 采用高层

LSTM-PPO、低层 GAT-PPO 的分层结构，为本项目提供了参考。原文研究动态到达、优先级和双队列问题；本项目将其适配到离线 DAG、前驱约束和 makespan 目标。

表 1-2 方法背景与本项目采用范围

方法背景	原有思想	本项目采用范围
HEFT	异构环境中的任务排序与预计完成时间放置	同语义基线、启发式先验与候选保留
PPO 及分层策略	从采样轨迹优化策略；分解复杂动作	高层选择任务、低层分配节点；两层独立更新器
LSTM 与 GAT	序列编码和图注意力表示	高层编码当前任务特征序列，低层编码任务条件下的资源图
图学习与多智能体比较	联合动作建模及协同决策	GraphPPO、TierMAPPO 为本项目适配实现，
搜索与候选择优	生成、修复并比较多个可行方案	独立搜索与 HRL 完整候选分别评分，最后统一校验和选择

比较这些方法时，任务图、执行时间、通信、约束和指标定义必须一致。项目使用 GrapheonRL 派生数据，并按基础 DAG 固定划分数据集。不同论文中的原始数字缺少这样的共同条件，不宜直接并列比较。数据来源、许可和适配过程见第 2 章，实验协议见第 4 章。

项目据此确定了三项工程需求。统一场景和调度语义，避免不同约束影响方法间的公平比较。提供可训练接口和明确的阶段控制，让采样、更新和权重选择可以追踪。保留完整合法候选，用确定性模拟评分检查调度质量。

正文中的“HRL 组合搜索”是演示简称，指 HEFT、独立搜索和冻结 HRL 依次生成完整调度后择优。HRL 不参与 beam 的分支展开，目前三路候选也按顺序执行。只要候选都成功完成，并采用同一确定性目标，组合就能保留最小 makespan。这一性质不保证全局最优、更快的推理或真实系统中性能不下降。

表 1-3 项目工作与后续章节

项目工作	具体内容	论证位置
统一框架与任务适配	数据资源适配、合法动作、分层策略及统一接口贯通	第 2 章方法与第 3 章实现
候选改进与组合验证	实际关键链和关键块修复、独立候选评分及最终重放	第 2 章流程与第 4 章结果

项目工作	具体内容	论证位置
训练评测与贡献 诊断	基础 DAG 隔离、三种子记录、检查点选择与 分支归因	第 4 章证据与第 5 章难点

这些工作属于项目实现和任务适配的贡献。第 6 章将结合前文证据总结完成情况和后续工作。

第 2 章 技术理论

本项目面向云边端异构资源上的离线 DAG 调度，将一次决策拆分为任务选择和节点分配。系统通过统一场景、执行与通信模型、合法动作和插入式模拟器，连接数据处理、策略训练与评估。除分层强化学习外，系统还提供独立搜索与 HEFT 候选，校验完整调度后择优。学习策略提出任务和节点动作，模拟器计算可执行时间，组合模块比较候选，各模块职责明确。

本章介绍第 4 章实验采用的模型和算法语义。LSTM、GAT、PPO 和 HEFT 均基于已有技术思想。项目贡献主要体现在问题适配、约束语义统一、候选修复和可追溯的集成流程。具体质量、耗时和学习增量见第 4 章的分项结果。

2.1 设计思想

2.1.1 问题建模与统一调度语义

依赖任务表示为有向无环图 $G=(V,E)$ 。任务 i 包含工作量，以及可选的 CPU/GPU/内存需求和设备要求。边 (i,j) 表示任务 j 依赖 i 的输出，边数据量决定跨节点传输开销。资源集合包括计算节点、设备速度、容量和成对带宽，必要时还可提供成对时延。调度结果记录每个任务的节点、开始时刻和结束时刻。优化目标是所有任务的最晚完成时刻，即 makespan。

$$C_{max} = \max_{i \in V} F_i, F_i = S_i + p_{i,n(i)} \quad (2-1)$$

其中， $n(i)$ 为任务 i 的执行节点， S_i 和 F_i 分别为开始和结束时刻， p_{iu} 为任务 i 在节点 u 上的执行时长。 p 由 ExecutionTimeModel 明确计算。普通速度模型用工作量除以计算速度；GrapheonRL 场景按配置读取 device_speeds，使用任务设备对应的速度。设备能否执行任务由独立约束检查判断，不能用一个无条件速度公式概括所有设备的执行语义。

MatrixCommunicationModel 将同节点传输时间设为 0。跨节点传输时间由边数据量除以成对带宽，再加上可选链路时延得到。成对带宽是数据适配后的模型输入，当前系统尚未模拟路由竞争或网络拥塞。

$$\tau_{ij}(u,v) = 0 \quad (u=v); \tau_{ij}(u,v) = \frac{d_{ij}}{b_{uv}} + l_{uv} \quad (u \neq v) \quad (2-2)$$

这里， d_{ij} 为依赖边数据量， b_{uv} 为带宽， l_{uv} 为时延。对候选节点 u ，各前驱的完成时刻加上传输时间后，取最大值作为数据就绪时刻。无前驱任务取 0。模拟器随后在 u 的已有时间线中寻找最早空闲区间。该区间不能早于数据就绪时刻，长度至少为 p_{iu} ，其起点即 EST；EFT=EST+ p_{iu} 。插入调度可以利用时间线中间的空隙，也可以在节点队尾追加。

“任务可选”表示所有前驱已放入调度表，此时前驱在模拟时间上可能尚未执行完毕。后继的实际开始时刻仍受数据到达时间约束。节点只要满足容量和设备要求，就可以被选择。即使节点已安排其他任务，也可以通过时间线插入确定执行位置。当前模拟器为每个节点维护串行、不抢占的时间线。容量属性用于筛选可行节点，尚未用于模拟同节点多任务并发共享资源。

2.1.2 分层动作与合法性约束

每个任务都经历一次完整的“选任务—选节点—放置”过程。高层从尚未调度且全部前驱已放置的任务中选择，低层从满足所选任务容量和设备要求的节点中选择。高层确定任务后，再构造该任务对应的 EST、EFT 和通信特征。这样可以保持任务与节点适配关系明确。

表 2-1 状态 动作与环境转移

对象	主要信息或动作	实现约束
高层观测	任务工作量、入出度、前驱放置进度、就绪与已调度标志、拓扑层级、执行时间统计、数据量和 HEFT rank；资源特征	当前任务特征为 17 维； <code>task_mask</code> 表示有效任务位置， <code>ready_mask</code> 才是高层合法动作集合
高层动作	从 <code>ready_mask</code> 中选一个任务	已放置任务和前驱未全放置的任务被排除
低层观测	所选任务特征；节点速度、执行时间、通信时间、依赖就绪时间、EST、EFT、可用时刻和已分配任务数；资源图	当前节点特征为 8 维，资源边特征为带宽和时延相关信息
低层动作	从 <code>node_mask</code> 中选一个节点	已声明的容量与设备约束必须满足；忙碌状态不直接屏蔽节点
转移与终止	提交一个合法放置，更新时间线和观测	每次放置一个任务；全部任务放置后终止；非法动作显式报错

两层均使用掩码分类分布。非法位置的 `logit` 被设为极小值，策略随后从合法动作中采样，或选择概率最大的动作。PPO 保存采样时的观测、`mask`、动作、旧对数概率和值估计，更新时复用同一 `mask`。这样，新旧策略的概率比就在相同可行域上计算。环境执行和候选重放时还会再次检查约束。合法性主要由明确规则保证，无需假定网络已学会所有约束。

正式配置以部分 `makespan` 增量的负值作为奖励，并用同场景 HEFT 完成时间归一化。设 M_t 为第 t 次放置后的部分 `makespan`， $M_0=0$ ， C_H 为 HEFT 参考值，则：

$$r_t = -\frac{M_t - M_{t-1}}{C_H}; \sum_{t=1}^{|V|} r_t = -\frac{C_{max}}{C_H} \quad (2-3)$$

完整轨迹中的增量相互抵消，未折扣累计奖励等于最终负归一化 **makespan**。正式配置中，两层 γ 均为 1。联合训练时，同一次放置的奖励分别进入两层轨迹；两层奖励相加并不代表一个新的双倍目标。通信惩罚项权重为 0，势函数奖励关闭。代码支持这些可选功能，但本批次没有使用。

2.1.3 模块解耦与同场景评估

数据适配器将不同输入转换为 **Scenario**，保留数据来源、基础 **DAG** 标识和资源配置。场景预处理完成静态图缓存和输入校验。执行模型、通信模型和模拟器负责时间与约束语义，不依赖神经网络。环境在模拟器上构造观测、奖励和动作 **mask**。策略通过统一的 **reset**、**select_task**、**select_node** 接口接入评估器。

统一接口让 **HEFT**、纯 **HRL**、搜索、组合和外部比较策略能够在相同输入上运行。每路独立候选各自建立环境，生成完整调度，只复用当前场景已校验的 **HEFT** 参考标量。共享分母并不替代任何一路候选的实际执行。公平比较还要求执行模型、通信模型、插入规则和资源实现一致，仅有相同策略名称并不够。

2.2 技术路线

2.2.1 两级策略网络与 PPO 更新

高层先对任务特征做线性映射和 **tanh** 激活，再用单层 **LSTM** 编码，并将每个任务位置的隐表示映射为任务 **logit**。高层 **critic** 对有效任务表示池化，拼接平均资源特征后估计状态价值。当前 **LSTM** 按 **Scenario** 中的源任务顺序接收输入。拓扑层级参与特征编码，但缓存的 **topological_order** 未用于明确重排输入。每次决策都会重新编码当前序列，**LSTM** 隐状态不跨调度决策传递。它并非持续记忆历史调度过程的循环控制器。

低层 **GAT** 编码计算资源图。第一层采用四头注意力，对各头输出取平均；第二层采用单头注意力。注意力分数包括节点相关项和资源边特征的线性映射。资源表示、所选任务编码和节点时序特征随后拼接，得到每个节点的 **logit**。低层 **critic** 对资源表示池化，拼接任务特征后估计价值。任务 **DAG** 依赖通过观测和合法动作影响决策，这里的 **GAT** 并非高层任务 **DAG** 编码器。

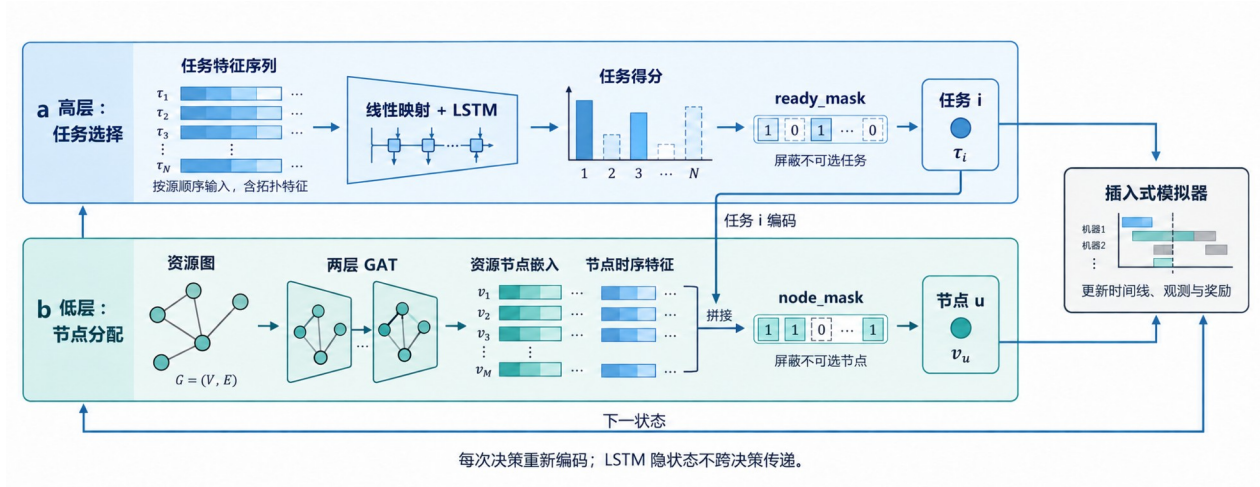


图 2-1 展示动作决策路径。高层选定任务后构造低层观测，GAT 编码资源图。模拟器更新后，模型重新编码任务序列，LSTM 隐状态不跨决策传递。

两层 actor-critic 分别使用 PPO 更新，算法思想参考 Schulman 等（2017）。令 ρ_t 为新旧策略对采样动作的概率比， $\hat{A}_t$ 为 GAE 优势估计，裁剪目标为：

$$L_{clip} = E_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)] \quad (2-4)$$

实现中最小化负载剪目标、价值误差和负熵正则的加权和。同时使用优势标准化、梯度范数裁剪、可选价值裁剪和 KL 提前停止。GAE 根据各层自己的价值估计计算，联合阶段也不共用 critic。不同任务数的样本按张量形状分组，确保可变规模观测和对应 mask 一起进入更新。

表 2-2 正式批次的主要模型与 PPO 参数

配置项	高层	低层
编码器	LSTM，隐维度 128	两层 GAT，隐维度 64，注意力头数 4 和 1
初始学习率	0.0001	0.00005
折扣与 GAE	$\gamma=1$ ， $\lambda=0.95$	$\gamma=1$ ， $\lambda=0.95$
PPO 更新	clip=0.1；每次更新最多 2 轮；batch_size=128	同左；并非每个阶段都启用低层更新
正则与稳定化	熵系数 0.001；target_kl=0.01；价值裁剪 0.2；最大梯度范数 0.5	同左

这些参数来自正式批次保存的 `residual_hrl` 配置，仅代表该实验设置，未证明是所有场景的最优超参数。LSTM 与 GAT 分别参考 Hochreiter 和 Schmidhuber (1997)、Veličković 等 (2018)。

2.2.2 启发式残差与训练阶段

两层策略在启发式偏好上学习有界修正。高层 prior 为归一化 HEFT upward rank，低层 prior 为负归一化 EFT。神经网络对 `logit` 的修正经 `tanh` 限幅。正式配置的启发式权重为 8，残差幅度上限为 0.5：

$$\text{logit}_{high,i} = 8 \times \overline{rank}_i + 0.5 \times \tanh(z_{high,i}) \quad (2-5)$$

$$\text{logit}_{low,u} = -8 \times \overline{EFT}_{i,u} + 0.5 \times \tanh(z_{low,u}) \quad (2-6)$$

其中， z 是网络未经限幅的输出，`rank` 和 `EFT` 均按环境定义归一化。`actor` 末层采用零初始化，初始神经修正为 0。确定性推理对零残差输出还提供 `float64` 启发式路径，以减少浮点排序差异。此时策略主要继承启发式偏好，初始模型的调度能力尚不能归为学习成果。`tie-break` 和具体策略执行规则也需要单独核对。

当前配置只执行两个训练阶段。前 864 episodes 为 `high_only_eft`：高层按 PPO 策略采样并更新，低层采用固定最小 EFT 规则，低层网络和优化器冻结。后 1728 episodes 为 `joint`：两层分别采样、收集轨迹并更新。冻结时除跳过 `optimizer.step` 外，还会禁用梯度、保持评估模式，并停止推进该层学习率调度器。

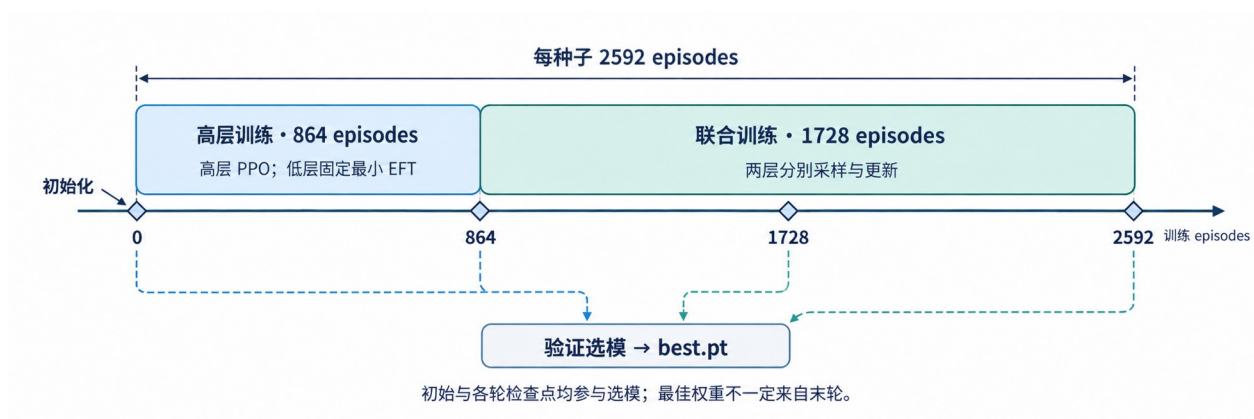


图 2-2 中，实线表示训练阶段顺序，虚线表示验证结果参与选模。step 0、864、1728、2592 均参与 best.pt 的选择，最优权重可能来自训练中途或初始模型。图中只包含本批次启用的高层训练和联合训练。

每个模型在每个种子下训练 2592 episodes，三个种子独立运行。第 4 章结果显示，HRL 在 seed 2026 选择 step 864，在 seed 2027 和 2028 选择 step 0，后期验证表现有所退化。

2.2.3 HRL 搜索和候选验证选择

搜索分支使用 HEFTSafePortfolioPolicy。它保留 HEFT，并生成 Arabnejad 和 Barbosa（2014）提出的 PEFT 相关候选、通信权重变体、有限偏差和随机扰动等多种完整候选，随后执行关键任务局部改动、beam 扩展和关键块修复。HEFT 参考 Topcuoglu 等（2002）。这些启发式候选用于提出调度，最终优劣由真实配置下的统一模拟器评分。候选生成时的加权估计有别于最终 makespan。

beam 扩展用完整调度的“决策顺序—节点分配”标识去重。每轮选择分数较好且尚未展开的候选，再尝试为关键任务更换节点或调整决策顺序。暂时没有直接改善的分支也可以继续扩展，所以它并非一次无改善就停止的纯贪心下降。前缀缓存复用改动位置之前的模拟状态，共享静态缓存并复制动态时间线。该机制可以减少重复计算，但提速比例仍需同环境对照验证。

关键块从最终完成的任务向前回溯一条实际关键链，同时考虑依赖数据到达和同节点前序任务的阻塞。算法从关键链中取短连续片段，寻找共同可行节点，并按估计成本筛选共同迁移方案。随后保留后缀，或用贪心方法修复后缀，生成完整调度。关键块估计只用于筛选，收益仍由重放后的 makespan 判断。原 DAG 的静态最长路径与资源分配后的实际关键链需要区分。

表 2-3 正式搜索配置及停止边界

组成	已启用参数	预算或停止规则
初始候选	PEFT 权重 0、0.25；通信倍率 0、0.25、2、4；扰动候选 16；节点偏差候选 1	CPOP 关闭，任务偏差候选为 0；扰动 task/node top-k 均为 2，替代节点概率 0.15
beam 局部扩展	最多 3 轮；宽度 3；关键任务数 8；节点替代数 2；任务替代数 1	没有未展开候选时结束；相同完整调度去重，保留已有最优候选
关键块修复	最多 2 轮；每轮筛选块数 4；块最大长度 3；节点替代数 2	无候选或本轮最优结果未较 incumbent 改善超过 10^{-12} 时提前结束

HRL 搜索模块向搜索分支传入 `learned_policy=None`。HEFT、搜索和 HRL 按顺序分别完成调度，再比较通过校验的候选。GraphPPO 和 TierMAPPO 用于外部比较，不属于组合内部的分支。

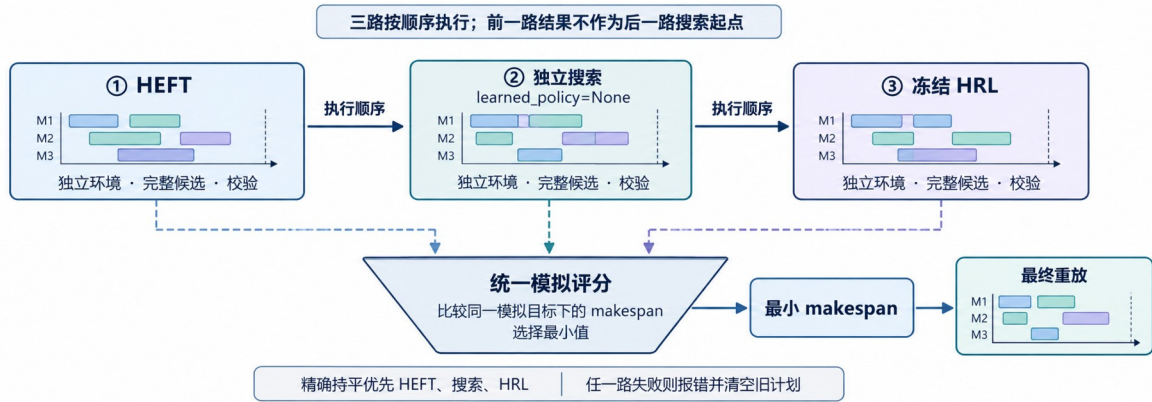


图 2-3 中，实线表示 HEFT、搜索、HRL 的串行执行顺序，虚线表示完整候选汇入比较。各路独立生成并校验，前一路结果不作为后一路的搜索起点。系统统一评分择优，再最终重放。

$$C_{combined} = \min(C_{HEFT}, C_{search}, C_{HRL}) \quad (2-7)$$

该关系要求三路候选均完整、合法，并采用同一确定性模拟器和评分目标。精确相等时，按 HEFT、搜索、HRL 的顺序优先选择。统计改善所用的浮点容差与精确 tie-break 需要区分。若候选失败或参考分母不一致，当前实现会明确报错并清空旧计划，不会忽略失败候选后报告成功。

候选最小值表明，组合在当前模拟目标上不劣于已完成候选。这并不保证全局最优、更快的推理或真实部署中性能不下降。组合耗时应包括所有候选的生成、校验、选择和重放成本。

算法 2-1 独立组合的核心过程

输入：场景 S ，HRL 权重 θ ，独立搜索配置 B

清空旧候选和旧重放计划

按顺序对 HEFT、Search(B , learned_policy=None)、HRL(θ):

 在相同语义的新环境中生成完整调度与决策序列

 校验任务覆盖、约束、时间线和有限正 makespan

 校验各分支的 HEFT 参考分母一致；失败则报错

按 makespan 取最小值；精确持平优先 HEFT、搜索、HRL

保存选中决策序列

逐动作检查 ready_mask 与 node_mask 后在最终环境重放

输出：调度结果、候选分数、选择来源及可追溯记录

2.3 代码原创说明

项目实现说明分别讨论算法思想、直接软件依赖、外部数据、训练产物，以及可能复制或改编的源码。当前源码中可以找到场景建模、合法动作、模拟器、两级网络、PPO、候选修复和评估的实现。这些内容说明了本地实现范围，但文件存在本身不足以认定全部源码原创。基础算法和通用软件包也不属于本项目独创。

Yue 等（2026）提出的 CPN-HRL 研究动态任务到达、优先级、截止期和双队列机制，采用高层 LSTM-PPO 与低层 GAT-PPO 分解决策。本项目参考这一分层思路，为离线 DAG 建立前驱约束、插入时间线和 makespan 奖励。原文的动态双队列、多目标收益和实验数字未沿用。

策略梯度的基础可追溯到 Williams（1992）提出的 REINFORCE 方法，通过采样回报估计策略参数的梯度。Schulman 等（2015）提出广义优势估计（GAE），利用价值估计与时间差分残差构造优势，调节估计的偏差与方差。本项目在 PPO 更新中使用 GAE；该理论来源与分层训练、动作掩码和阶段冻结等本地工程实现应分别说明。

图表示学习方面，Kipf 和 Welling（2017）提出图卷积网络，通过邻域传播结合节点特征与图结构；Hamilton 等（2017）提出 GraphSAGE，通过采样和聚合邻域特征学习归纳式节点表示。这两项工作为理解资源图表示提供背景。本项目低层网络采用 Veličković 等（2018）的 GAT 思想，具体的任务条件特征、资源边特征和动作约束见 2.2.1。

学习式调度方面，Mao 等（2019）提出 Decima，将作业依赖图表示与强化学习结合，学习数据处理集群的调度策略。该工作说明了利用图结构学习调度决策的研究路径。其面

向持续到达的作业和平均作业完成时间，本项目面向离线 DAG 的 makespan；两者的任务条件与评估目标不同，原文实验数值不能直接作为本项目的性能结论。

表 2-4 实现范围与来源核查

模块或材料	实现方式与项目工作	来源及版本状态
scenario、scheduling、env	本地模块统一场景、执行通信语义、插入调度和合法动作	以当前提交源码及冻结哈希核对；列表调度思想参考既有工作
high_lstm、low_gat	使用 PyTorch 实现两级 actor-critic、任务条件资源图与有界启发式残差	LSTM、GAT 原始研究；CPN-HRL 分层思路；本地网络代码
ppo、phases、分层训练器	本地实现掩码轨迹、GAE、裁剪更新和阶段冻结	PPO 原始研究；正式参数见批次 residual_hrl 配置
portfolio、hybrid	本地实现候选集合、关键链与块修复、独立组合和重放	HEFT、PEFT 等既有调度思想；当前组合文件存在冻结版本差异
GrapheonRL 派生数据与适配器	读取 STG JSON 与资源配置，转换设备速度、成对带宽和来源字段	本地数据登记 DOI 10.5281/zenodo.18927122；适配器版本 grapheonrl-stg-json-v2-archive
best.pt 等模型权重	实验训练及验证选择产生；保留初始 step 0 的可能	三种子 provenance 和 checkpoint 哈希提供实验来源

直接依赖按 pyproject.toml 登记。核心依赖为 NumPy、NetworkX、PyYAML；训练依赖 PyTorch；分析依赖 pandas、Matplotlib、SciPy；开发测试依赖 pytest。声明的版本下限有别于正式实验锁定版本，精确版本以批次 pip_freeze.txt 为准。发布前应逐项核对实际发行包的许可证，并保留相应声明。数据包的 CC-BY-4.0 不自动适用于这些软件或整个项目。

本轮源码核对中，模拟器、环境、两级网络、PPO、phases、HRL 策略和 portfolio 八个重点文件与正式批次 source_sha256.json 相符，也与提交副本一致。hybrid.py 与提交副本一致。

第 3 章 项目介绍

CPN-HRL-DAG 将离线任务调度分为数据准备、场景配置、策略运行、结果验证和可视化五个环节。使用者可以通过同一场景和策略接口，运行启发式、学习模型或独立组合，查看任务分配和时间线。结果可沿配置、权重和逐场景记录追溯。本章介绍模块职责和使用方法，模型与搜索原理见第 2 章，完整实验结论见第 4 章。

项目提供 Python 训练评测工程和离线浏览器演示。训练评测工程用于数据准备后的训练、评估和证据保存，浏览器演示用于查看或重新计算四个固定样例。当前系统在确定性模拟环境中进行调度。

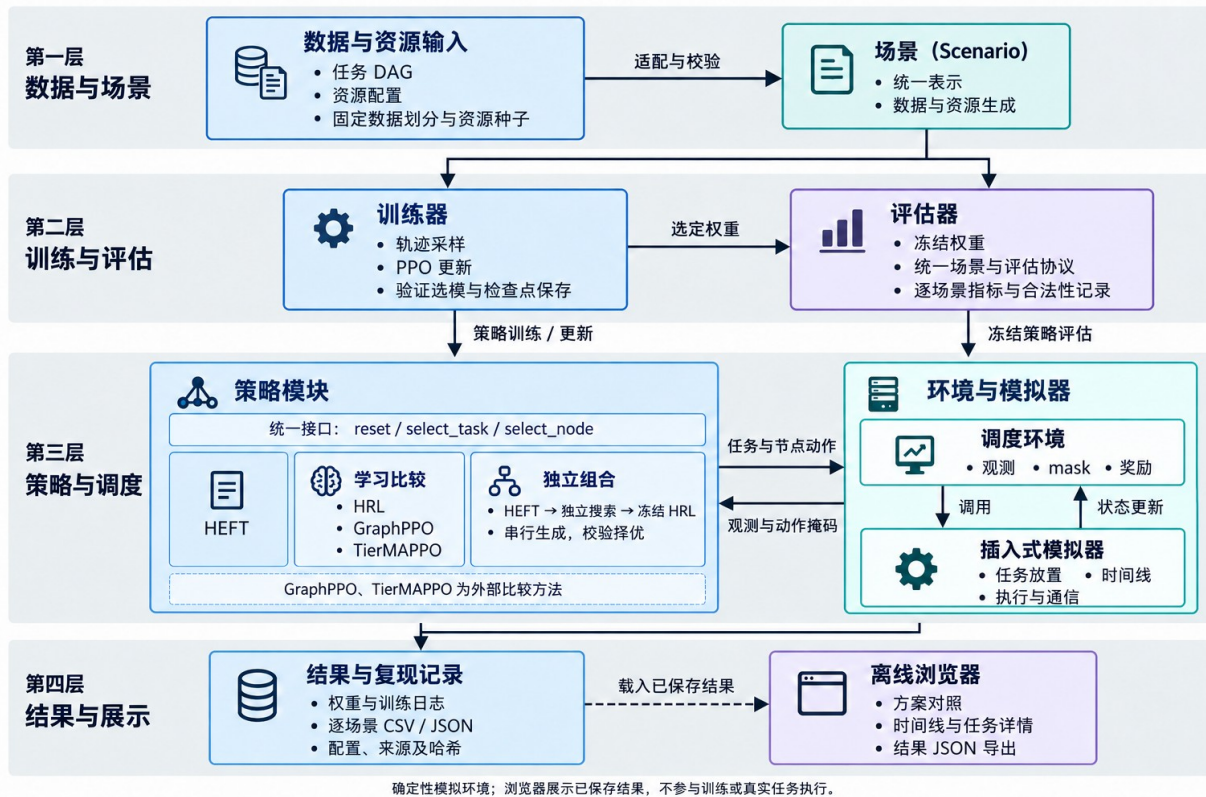


图 3-1 展示整体架构。数据和资源经适配后形成统一 Scenario。训练器和评估器调用策略及调度内核，保存权重、逐场景结果和复现记录。实线表示主要调用或数据传递关系，虚线表示已保存的演示结果载入离线浏览器。

3.1 功能介绍

3.1.1 场景与资源配置

数据适配器将外部任务图和系统配置转换为统一 Scenario。该对象包括任务、依赖边、计算节点、通信参数和来源元数据，策略无需直接读取原始归档格式。GrapheonRL 适配

器读取本地 STG JSON 和系统配置归档，并保留数据来源、原始图标识及适配器版本。这些信息用于检查划分和追溯结果。`prepare_data.py` 检查已准备的数据，不自动下载数据集。

数据先按基础 DAG 身份划分，再生成资源变体。同一拓扑更换资源后，仍保留 `original_graph_id`。训练与验证的隔离检查以这一身份为依据，避免将同一任务图的不同资源版本视为独立图。正式实验的场景规模和划分见第 4 章。新增资源配置后，应重新保存配置和实现种子，原实验统计不能直接沿用。

表 3-1 场景输入与资源替换

输入或配置	主要内容	使用结果
任务与依赖	工作量、设备及容量要求、前后继关系、边数据量	统一任务图；执行前检查图与字段合法性
原生资源	数据集节点、设备速度、容量与通信参数	<code>resources.mode=dataset</code> ，保留数据集资源
生成资源	云边缘节点数量、速度和容量区间；分层带宽与时延	<code>mode=generated</code> ，用固定种子生成资源并替换原配置
混合资源	同时保留原生与生成资源版本	<code>mode=hybrid</code> ，将生成场景追加到所属划分
来源与划分	原图标识、适配器版本、固定 <code>manifest</code> 、资源种子	支持按基础 DAG 检查隔离及定位结果来源

`materialize_resources` 调用资源配置解析和 `realize_scenario`，完成资源生成。`generated` 或 `hybrid` 模式必须提供 `resources.config`，每个划分至少提供一个实现种子。不支持的模式或空种子集合会报错。配置区间用于生成模拟资源，尚不能说明平台已探测到对应硬件。

下面摘录 `configs/generated_smoke.yaml` 的资源设置，说明资源替换入口。它生成两个云节点、一个边节点和一个端节点，完整文件还定义了容量、带宽和时延。该文件用于小规模流程检查，正式实验应使用对应的实验配置。

```
resources:
  mode: generated
  train_seeds: [101]
  validation_seeds: [201]
  test_seeds: [301]
  config:
    config_id: generated_smoke
    cloud: {count: [2, 2], speed: [8.0, 8.0]}
    edge: {count: [1, 1], speed: [4.0, 4.0]}
    end: {count: [1, 1], speed: [2.0, 2.0]}
```

3.1.2 模拟器和策略接入

系统将策略决策和时间计算分开。ScheduleSimulator 维护任务放置和节点时间线，执行模型与通信模型提供计算和传输时长。CloudEdgeEndDAGEnv 在此基础上提供观测、动作掩码、奖励和终止状态。Evaluator 通过统一策略接口调用不同方法，各类策略遵循相同的任务执行规则。

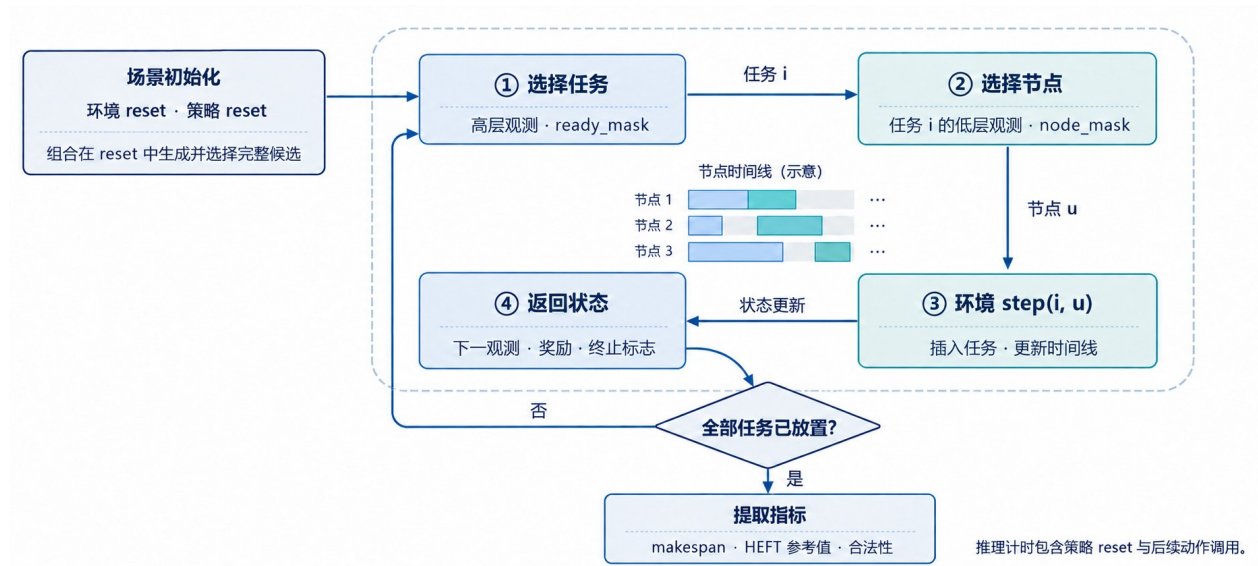


图 3-2 展示策略与环境的交互流程。每次先选任务，再根据该任务的观测和掩码选节点。环境调用插入式模拟器，返回下一状态。全部任务放置后，提取指标和合法性。组合在 reset 中生成并选择完整候选，随后通过同一接口重放，这部分成本计入推理时间。

表 3-2 统一策略接口

接口	输入与职责	输出与约束
reset	接收 Scenario，重置策略状态或生成候选计划	每个新场景重新调用；组合候选生成成本计入推理时间
select_task	高层观测、ready_mask、确定性标志	返回任务位置；须处于当前合法集合
select_node	所选任务对应观测、任务位置、node_mask、确定性标志	返回节点位置；须满足该任务的资源约束

评估首先初始化环境和策略，再循环执行“获取 ready_mask—选择任务—构造低层观测—选择节点—环境放置”。全部任务完成后，评估器提取 makespan、HEFT 参考值和合法性。忙碌节点仍可被合法选择，具体开始时刻由插入模拟器决定。非法动作和异常截断会单独处理，不作为正常结束。

新增策略实现上述三个接口后，即可复用统一评估流程。返回值必须使用当前 `Scenario` 的位置索引，`reset` 也要清理上一场景的状态。`HEFT`、`HRL` 和图学习比较策略使用相同环境。独立组合在内部依次生成 `HEFT`、搜索、`HRL` 三路完整候选，校验后保存最优决策序列，再通过同一接口重放。

当前可插拔接口适用于离线调度语义。新增能耗、动态到达或抢占目标时，还需同步修改场景、环境和评估协议。仅替换策略名称不足以实现这些功能。

3.1.3 训练评测与结果追溯

训练入口读取配置和固定划分，构建环境、策略网络和更新器，并保存训练日志、检查点和验证记录。正式批次的阶段和参数见第 2 章。

`run_server_comparison.py` 统一组织三模型训练和比较。运行前，它检查固定划分与协议，生成各模型配置，再启动训练和后续评估。默认三个完整覆盖轮次对应每模型 2592 episodes。该入口执行六方法比较，独立三候选组合评估则使用另一个入口。需要独立组合结果时，应运行 `evaluate_independent_hybrid.py`，并指定本轮配置和 `HRL` 权重。

表 3-3 结果文件与核对用途

产物	记录内容	核对方式
配置与运行计划	seed、数据划分、训练预算、搜索设置、设备安排	确认本轮实际配置，区分示例与正式运行
检查点与训练日志	权重、训练状态、验证点及更新记录	确认模型来源、是否训练完成、best 选自哪一步
逐场景 CSV 与汇总 JSON	场景身份、规模、策略、完成时间、参考值、合法性、耗时	从逐场景数据重算汇总，避免只保留一个平均数
候选与重放记录	分支分数、选择来源、任务节点决策及时间线	检查组合贡献和重放一致性
环境与哈希记录	软件版本、源码、数据和模型标识	判断复现输入是否与原始运行一致

推理计时包括 `policy.reset` 和后续动作调用，搜索在 `reset` 中生成候选计划的时间也计入其中。演示产物还分别记录组合推理、外层环境准备和额外独立重放审计的时间。读取界面耗时前，应查看 `provenance`，确认计时范围。不同机器、线程设置和计时范围下的数值不宜直接比较。

运行脚本通过状态文件和日志记录完成情况。独立组合评估和演示入口拒绝覆盖非空结果目录。演示运行先写入 `running`，完成后写入 `complete`，异常时写入 `failed` 并保留错

误信息。历史记录若仍为 `running`，只能说明缺少完整结束记录，文件时间不足以判断当前进程是否仍在运行。

3.1.4 运行部署与交付

提交目录中的 `src/` 是可独立安装的工程根目录，其中 `src/cpn_hrl_dag/` 才是 Python 包演示启动器位于提交目录根部，可根据自身位置找到资源。命令中的相对路径按对应工作目录解释。

表 3-4 提交目录摘要

路径	内容	使用范围
<code>src/</code>	Python 工程、配置、脚本、部署文件、划分及测试	安装、训练、评估和平台验收
<code>demo/</code>	六场景、三份学习模型权重、哈希清单、网页与核验结果	无需重训的本地演示
<code>tests/</code>	提交包核对与演示验收记录、测试入口	区分文件校验、功能测试及界面检查
<code>docs/ 与 presentation/</code>	说明书等文档及展示材料的交付位置	项目说明和展示

查看已生成的界面时，直接打开提交包 `demo/index.html` 即可，无需启动训练。重新计算时，先在提交目录根部安装工程，再运行启动器。默认输出到 `demo/runs/` 下的时间戳目录。子进程成功后，启动器打开新结果页；添加 `--no-open` 可关闭自动打开。

```
python -m pip install -e "./src[train,analysis,dev]"
python run_demo.py --no-open
```

当前提交版启动器调用 `src/scripts/run_hybrid_demo.py`。它除生成组合内部候选外，还加载 `GraphPPO` 和 `TierMAPPO` 权重，形成四方法比较。原项目根目录中的同名脚本只生成组合候选演示，使用时应确认脚本位置。提交版先验证资产哈希、`seed` 和基线训练状态并核对比较协议。指定样例时可添加 `--scene` 和完整场景标识，未知标识会报错。

完整训练需要另行准备原始数据。演示仅打包六个场景和必要权重，未包含完整训练数据及全部历史 `outputs`。以下命令在 `src/` 工程根目录执行。先检查本地数据，再预览 CPU 训练计划。实际运行时移除 `--dry-run`，并使用新的输出目录。GPU 运行需明确选择可用且获准使用的设备编号。

```
python scripts/prepare_data.py
python scripts/run_server_comparison.py --device cpu --seed 2026 --output
outputs/reproduce_seed2026 --dry-run
```

独立组合评估以 `configs/hrl_independent_hybrid_seed2026.yaml` 为入口示例。执行前应确认其中的权重路径、划分和搜索配置符合预期实验。复制工程后，历史输出路径仍需另行准备。下列命令只评估固定 `validation`，不启动训练。

```
python scripts/evaluate_independent_hybrid.py --config
configs/hrl_independent_hybrid_seed2026.yaml --output
outputs/hybrid_reproduce_seed2026
```

平台适配保留 `openEuler 24.03 LTS-SP4` 容器配方，并新增 `openEuler` 与 `openKylin` 的 `x86_64` 虚拟机运行记录。两系统均使用 `CPU` 单线程配置，完成短训练和验证集评估。`openEuler` 验收入口依次保存环境信息，检查依赖和数据，执行测试、短训练、权重重载与 `HEFT` 验证。以下命令适用于已准备好的 `openEuler` 虚拟机，输出目录应尚不存在。`openKylin` 的实际配置与结果见第 4 章。

```
python scripts/verify_platform.py --output outputs/platform/run_new --execution-
kind vm
```

2026 年 9 月 30 日的 `openEuler` 虚拟机验收总状态为 `passed`，六个阶段均通过，训练后评估与新进程重载结果一致。`openKylin` 也已保存短训练、`HRL` 和 `HEFT` 验证结果，其 `README` 记录测试与重载完成。两系统各评估 108 个场景，调度合法率均为 100%。这些记录验证了对应虚拟机中的 `CPU` 训练评测流程。

3.2 软件界面

3.2.1 场景选择与方案对照

离线网页提供六个样例，覆盖 50、100、300 个任务以及同构、异构资源。每组按场景标识字典序选取首个验证样例。界面支持按规模和资源类型筛选，再选择具体场景。四个比较方法为 `HEFT`、`GraphPPO`、`TierMAPPO` 和 `HRL` 组合搜索。搜索与纯 `HRL` 属于组合内部候选，界面的四方法比较并非四路组合。

演示清单记录，三份学习权重均来自正式批次 `seed 2026`，`HRL` 检查点为 `step 864`。页面展示已计算并保存的结果，播放按钮只移动时间游标。重新推理需要运行 `Python` 启动器，并生成新的结果目录。

a 场景筛选与组合指标



b 四方法对照与资源信息



c 已保存方案的调度时间线



图 3-3 展示同一归档界面的分区截图：a 为场景筛选与组合指标，b 为四方法比较与资源信息，c 为已保存方案的局部时间线。场景为 grapheonrl:rnc300:rand0000:homogeneous，选中 HRL 组合搜索。300 个任务、三节点、ratio 0.996636 和 31.390 s 均对应这一样例。截图只做裁剪和重排，数值未改动。图来自 tests/comparison_preview.png，数据来自 demo/verified_comparison/results.json。

页面顶部卡片始终显示当前场景的组合指标，包括相对 HEFT 改善、归一化完成时间组合推理耗时和四方法合法性。点击比较表中的其他方法，会切换时间线、DAG 着色和任务详情，顶部卡片仍显示组合指标。阅读时应同时确认表中高亮行和时间线标题。

3.2.2 调度查看与结果导出

甘特图按计算节点分行，用任务开始和结束时间绘制区间。选择任务后，可以查看工作量、前驱关系及当前方案中的放置位置。DAG 视图按依赖层级展示任务，并突出与当前任务相关的边。这些图用于解释完整调度方案，尚不反映真实任务执行状态。

表 3-5 界面操作与核对内容

操作	查看内容	解读边界
筛选规模与资源	当前场景身份、任务数、节点属性	同一规模不代表同一拓扑；确认完整场景标识
点击方案行	对应时间线、DAG 颜色和任务详情	顶部卡片仍为组合指标，比较方法指标看表格
点击任务与播放	任务放置、依赖关联及时间游标	播放已有时间线，不表示实时推理或任务执行
导出结果 JSON	页面内完整结果对象	导出包含全部载入场景，不只当前筛选项

导出按钮将页面内的数据对象保存为 `hybrid_demo_results.json`。完整运行目录还包含 `metrics.csv`、`summary.json`、`provenance.json`、`progress.jsonl` 和 `status.json`。结合权重与场景哈希，可以检查结果来源。页面以单文件形式嵌入数据，图表和导出共享同一数据对象，数值应从数据文件读取。

提交包已有演示验收记录为 `complete`，记录六个场景、四个方法、合法率 1.0、14 项演示测试通过，以及 24 次时间线检查。界面检查未报告页面错误。这些记录来自此前的演示验收，本章未重新训练或完成 108 场景评估。六个样例用于展示操作和调度差异。完整性能判断仍以第 4 章的固定验证集和三种子统计为依据。

第 4 章 实验测试

本章检查 CPN-HRL-DAG 在离线依赖任务调度中的功能正确性、调度质量和工程可复现性。测试包括约束与接口检查、固定验证集上的策略比较，以及 openEuler 和 openKylin 虚拟机中的 CPU 运行验证。三类证据分别回答调度是否合法、方法表现如何，以及训练评测流程能否在目标系统运行。

实验采用固定 108 个验证场景和三个随机种子。独立组合方法的平均归一化完成时间为 0.918223，较 HEFT 降低约 8.18%，归档调度记录全部合法。下文先介绍评测协议，再报告结果及适用范围。

4.1 测试说明

4.1.1 功能与约束测试

功能检查围绕统一 Scenario、合法动作掩码和插入式调度模拟器进行。任务在前驱已放入调度表后进入可选集合。模拟器仍依据前驱的实际完成时刻和通信开销，约束后继开始时刻。节点只要满足容量与设备约束，就可进入候选集合。忙碌状态由时间线插入过程处理。

表 4-1 功能检查对象与判据

检查对象	通过判据	主要证据
依赖与时间线	任务恰好调度一次；后继满足数据到达约束；同一执行资源上的任务无非法重叠	test_simulator.py; test_environment.py
动作与资源约束	可选任务、可行节点与环境执行一致；非法动作不能进入有效调度	test_ppo_mask.py; test_policy_interface.py
数据与配置	基础 DAG 跨划分隔离；资源参数可替换；适配后的执行与通信语义一致	test_dataset_split.py; test_grapheonrl_adapter.py
候选择优与重放	各候选独立完成；结果为合法候选最小值；重放检查任务就绪及节点可行性	test_independent_hybrid.py
失败与产物保护	无效候选和协议不一致显式失败；已有结果不被无提示覆盖；权重来源可追溯	test_independent_hybrid.py; test_platform_acceptance.py

上述测试文件位于项目 tests/。测试判据和通过记录分别保存。已有测试代码本身不足以说明当前交付版本全部通过，对应执行结果和版本范围见 4.2.1。

4.1.2 数据划分和性能协议

实验使用 GrapheonRL 派生数据，任务规模为 50、100 和 300，并为基础 DAG 构造同构与异构资源场景。固定清单采用 mixed_iid 协议，划分种子为 7，按基础 DAG 划分数据。训练集包含 432 个基础 DAG、864 个场景，验证集包含 54 个基础 DAG、108 个场景。另有 54 个基础 DAG 分配到 test，本批次未评估该划分。

同一基础 DAG 的资源变体放入相同划分，避免同一任务图的其他资源配置进入验证集。三个实验种子 2026、2027、2028 使用相同验证场景，是对同一组场景的重复运行。324 条种子与场景组合记录不能视为 324 个独立样本。

Henderson 等（2018）讨论了深度强化学习结果中的随机性、实验设置差异和复现困难，强调规范报告实验条件与性能变异。本项目据此说明随机种子、统一评测条件和统计口径，分别保存配置、权重与逐场景结果。三种子重复运行支持当前协议下的变异分析，验证集参与选模所带来的限制仍需通过独立测试处理。

表 4-2 数据与训练预算

项目	固定设置	统计含义
数据划分	训练 864 场景；验证 108 场景	432/54 个基础 DAG，各有两种资源配置
验证规模	50 任务 24 场景；100 任务 42 场景；300 任务 42 场景	每种资源配置各 54 场景；规模分组样本量不相等
训练预算	每模型每种子 2592 episodes，384900 次任务决策	三个学习模型均覆盖训练场景 3 轮
HRL 阶段	前 864 episodes 高层训练与固定 EFT；后 1728 episodes 联合训练	低层预训练为 0；验证间隔为 864 episodes
模型选择	评估初始模型和每轮检查点，保留验证最优模型	所选模型不一定来自最后一轮
总预算	9 次模型训练；23328 episodes；3464100 次任务决策	不包括搜索候选评估的计算量

记场景 i 上方法 m 的 makespan 为 $C(m,i)$ ，同场景 HEFT 的 makespan 为 $C(\text{HEFT},i)$ ，定义 $r(m,i)=C(m,i)/C(\text{HEFT},i)$ 。每个种子先对 108 个场景的 r 求算术平均，得到 $R(m,s)$ 。再对三个 R 求均值和样本标准差，标准差分母为 3-1。 r 越小越好，HEFT 的值固定为 1。

“较 HEFT 降低”按 $100 \times (1 - \text{三种子平均 } R)$ 计算。它表示归一化完成时间的改善，有别于原始完成时间总和、各任务平均耗时或真实业务时延的降幅。合法率按完整调度记

录统计；胜负与持平采用 `ratio` 容差 10^{-9} 。配对置信区间按基础 DAG 整组重采样，同一 DAG 的资源变体保留在同一组，采用 1000 次 `bootstrap`。该区间与三种子样本标准差具有不同统计含义。

各方法使用相同 `Scenario` 和模拟器，执行时间、通信和插入规则保持一致。场景覆盖与任务决策预算相同，并不意味着参数量、训练 FLOPs 或墙钟耗时相同。验证集参与选模，下文结果仅反映固定验证集表现，独立 `test` 泛化仍需检验。

4.1.3 系统环境与复现方法

正式性能批次在 Ubuntu 20.04.6 LTS 服务器上完成。归档记录包括 Python 3.11.7、PyTorch 2.6.0+cu124、CUDA 12.4 和 NVIDIA GeForce RTX 4080。模型使用 CUDA 训练，统一推理评估采用 CPU 单线程。新增双系统实验在 x86_64 虚拟机上使用 CPU 单线程短训练配置。服务器与虚拟机的依赖、训练预算和运行条件不同，耗时分别报告，不据此比较操作系统快慢。

表 4-3 环境记录与证据用途

环境	已记录配置	证据用途
正式训练与比较	Ubuntu 20.04.6 LTS; Python 3.11.7; PyTorch 2.6.0+cu124	三种子训练、检查点选择、统一策略比较
组合评估	CPU 单线程；同进程分支计时；候选依次执行	记录各候选质量、归因与组合端到端耗时
openEuler 虚拟机	24.03 LTS-SP4; x86_64; Python 3.11.6; PyTorch 2.9.1+cpu	188 项测试、短训练、重载及 HEFT 验证；验收状态 <code>passed</code>
openKylin 虚拟机	2.0 SP2; x86_64; Python 3.12.2; PyTorch 2.14.0+cpu	短训练与 HRL/HEFT 验证；测试和重载完成情况见 README

复现从项目根目录开始，先准备数据和冻结依赖，再检查划分清单与配置。`scripts/run_server_comparison.py` 为每个 `seed` 生成学习模型配置，完成三模型训练与六方法评估。`scripts/evaluate_independent_hybrid.py` 随后读取该 `seed` 的 HRL 最佳权重和搜索配置，完成独立组合评估。输出应使用新的空目录，并确认 `best.pt` 来自当前种子。

命令如下。GPU 编号应对应实际分配的设备。“新输出目录”和“本种子组合配置”是需要替换的路径参数。这里提供复现入口，本轮撰写未重新执行训练。

```
python scripts/prepare_data.py
```

```
python scripts/run_server_comparison.py --output 新输出目录 --gpus 0 --epochs 3 --
```

```
seed 2026 --threads 4
python scripts/evaluate_independent_hybrid.py --config 本种子组合配置.yaml
```

另外两个种子将 --seed 改为 2027 和 2028，并使用各自独立的目录。组合配置绑定 checkpoint、search_config、split: validation 和 output_dir。批次脚本还固定线程和随机性相关环境变量，并在种子之间校验源文件哈希。完整执行环境以归档 run_three_seeds.sh、pip_freeze.txt 和各次配置为准。

复现记录包括 source_sha256.json、split_manifest.json、config.yaml、best.pt、provenance.json、逐场景 CSV、候选决策序列和状态文件。

4.2 测试结果

4.2.1 功能完整性和平台验收

三种子验证归档中，各方法的记录均标记为有效调度。组合在每个场景上的完成时间，等于 HEFT、独立搜索和 HRL 完整候选的最小值。原验收报告核对了训练状态、配置、划分、HEFT 分母和权重哈希。检查范围限于保存产物和统计结果，未重新执行全部调度器，也未评估 test。

表 4-4 已有执行证据及验收边界

检查项	已有记录	可支持的结论
三种子策略评估	每 seed 六方法 648 条记录；组合评估另有 432 条候选与组合记录	各归档记录合法；候选择优满足最小值关系
openEuler 测试	虚拟机 pytest.xml: 188 项通过，0 失败、0 错误、0 跳过	对应虚拟机源码快照的测试通过
openKylin 测试	README 记录完成 188 项测试	归档说明记录测试完成
双系统短训练与重载	各 2 episodes、400 transitions；openEuler 重载一致性为 true	两系统均有训练及验证产物；openKylin 重载过程由 README 记录
openEuler 完整流程	虚拟机 acceptance.json 总状态及六阶段均 passed	依赖、数据、测试、训练、重载和 HEFT 验证完成
六场景演示	14 项测试通过；24 次界面时间线检查；无页面错误	六场景四方法的演示检查通过

新增结果来自 2026 年 9 月 30 日的两个虚拟机归档。openEuler 的验收总状态为 passed，六个阶段均通过。openKylin 保存了配置、训练汇总及 HRL/HEFT 逐场景结果；188 项测试和新进程重载完成情况由 README 记录。

openEuler 的内核为 6.6.0-159.4.16.170.oe2403sp4.x86_64，环境记录的容器标记为 false；openKylin 的内核为 6.6.0-25-generic。两份归档均对应 x86_64 虚拟机。旧 openEuler 容器复用 WSL2 内核，其 running 状态保留为历史记录。

两平台均使用 seed 2026 和同一 mixed_iid 划分，CPU 线程数为 1。短训练包含 1 回合 high_only_eft 和 1 回合 joint。高层 LSTM 隐藏维度为 16，低层 GAT 隐藏维度为 8、注意力头数为 2；PPO 每次更新 1 轮，batch_size 为 32。该配置小于表 2-2 的正式模型，训练预算也有明显差异。

表 4-4a 双系统短训练与验证结果

指标	openEuler 虚拟机	openKylin 虚拟机
训练回合 / transitions	2 / 400	2 / 400
所选检查点 step	2	2
优化器更新次数 高层 / 低层	14 / 10	14 / 10
验证场景 / 基础 DAG	108 / 54	108 / 54
HRL / HEFT 合法调度率	100% / 100%	100% / 100%
训练耗时 秒	5.606	5.215
HRL 重载验证 毫秒/场景	862.027	927.226
HEFT 验证 毫秒/场景	479.380	402.401

注：数值由 training_summary、验证汇总及逐场景 CSV 核对。训练耗时仅覆盖轨迹采集和 PPO 更新；HRL 推理耗时采用保存后的重载验证记录。两系统的依赖版本和源码提交不同，虚拟机资源也未按统一性能基准控制，表中耗时仅描述各次运行。

两个平台的 HRL 逐场景 makespan、HEFT 分母和 ratio 数值完全一致。openEuler 的训练后评估与重载评估也逐项一致。

两目录记录的七个输入压缩包 SHA-256 校验值相同。源码提交分别为 openEuler 的 33422a6f19f0 和 openKylin 的 dd91b62c32df，完整标识见各目录 source-commit.txt。openKylin 的训练汇总与独立 HRL 验证汇总一致。

4.2.2 总体性能与计算代价

表 4-5 汇总七种方法。GraphPPO 和 TierMAPPO 是本项目适配实现的比较策略。

表 4-5 固定验证集总体结果

方法	mean_ratio $\pm$ 样本标准差	相对 HEFT 降幅	平均推理秒/场景
HEFT	1.000000 $\pm$ 0.000000	0.0000%	0.175
GraphPPO	1.029373 $\pm$ 0.011873	-2.9373%	0.821
TierMAPPO	1.097279 $\pm$ 0.017729	-9.7279%	1.060
HRL 搜索	0.918223 $\pm$ 0.000498	8.1777%	5.297

HRL 搜索 mean_ratio 为 0.918223，平均归一化 makespan 降低 8.1777%。每个种子都有 106 个场景优于 HEFT、2 个场景持平，无退化。GraphPPO 和 TierMAPPO 的总体均值高于 1，说明这两种适配实现和训练配置在当前协议下尚未达到 HEFT 水平。

调度质量的改善需要额外计算。HRL 搜搜约 5.297 秒/场景，HEFT 约 0.175 秒/场景。分支计时包括 policy.reset 和完整调度执行，不含环境 reset。组合计时还包括各候选的环境准备、校验、选择和最终重放。两者计时范围不同，组合耗时不能仅按选中分支计算，其差额也不能全部归为神经网络开销。

4.2.3 规模与资源分组

三个任务规模组在每个种子中分别包含 24、42 和 42 个场景。先在种子内求各组场景均值，再对三个种子求平均。由于组内场景数不同，三个规模均值的等权平均不能代替总体成绩。表 4-6 从逐场景记录重新计算，并与已有规模归档交叉核对。

表 4-6 不同任务规模的平均归一化 makespan

方法	50 任务 24 场景	100 任务 42 场景	300 任务 42 场景
HEFT	1.000000	1.000000	1.000000
GraphPPO	1.059270	1.004680	1.036983
TierMAPPO	1.057991	1.059269	1.157739
HRL 搜索	0.915160	0.889808	0.948390

HRL 搜索在 50、100、300 任务组的均值依次为 0.915160、0.889808、0.948390，对应三种子样本标准差为 0.001910、0.000810、0.000992。100 任务组的平均相对改善最大。

表 4-7 不同资源配置的平均归一化 makespan

方法	同构 54 场景	异构 54 场景
HEFT	1.000000	1.000000

方法	同构 54 场景	异构 54 场景
GraphPPO	1.046634	1.012113
TierMAPPO	1.088935	1.105623
HRL 搜索	0.997167	0.839280

HRL 搜索在同构资源上的均值为 0.997167，在异构资源上为 0.839280，分别较 HEFT 降低约 0.28% 和 16.07%。对应种子标准差为 0.000098 和 0.001089。整体 8.18% 的收益主要集中在异构场景。资源配置可能同时改变节点数和设备组成，分组差异不能全部归因于单一异构性参数。

资源分组根据 `scenario_id` 的资源后缀识别，每个种子中各组有 54 条记录。同构与异构场景共享基础 DAG，是配对资源变体。表中保留六位小数，显示值相同并不意味着底层浮点结果完全一致。

4.2.4 学习过程和组件贡献

训练完成度和学习收益需要分别评价。九次模型训练都完成了既定预算，但 HRL 的检查点表现未显示调度质量随训练持续改善。表 4-8 列出初始模型和每轮验证值，同时呈现最佳结果与退化过程。

表 4-8 HRL 验证过程

评估 step	seed 2026	seed 2027	seed 2028
0	0.999908	0.999908	0.999908
864	0.997358	1.011029	1.000515
1728	1.002619	1.007827	1.015656
2592	1.006308	1.029647	1.042573
最终选择	step 864	step 0	step 0

seed 2026 选择了高层训练与固定 EFT 阶段结束后的模型，随后联合训练的已观测验证成绩退化。seed 2027 和 2028 均选择 step 0，即训练前的初始策略。三个种子的末轮表现都差于初始模型。训练流程完成并不意味着性能提升。现有记录尚无法确定退化的主要原因。

4.2.5 调度案例与局限

案例按固定、可复核的规则选择。在 seed 2026 的 50 任务验证场景中，按 scenario_id 字典序选择第一个异构场景，即 grapheonrl:rnc50:rand0000:heterogeneous。选择过程未按改善幅度挑选最佳案例。表 4-10 直接读取归档逐场景数值。

表 4-10 固定案例的候选完成时间

方法	makespan	相对 HEFT 的 ratio
HEFT	60.516626	1.000000
HRL 搜索	60.456521	0.999007

该案例中，HRL 搜索采用搜索候选，makespan 减少约 0.060105 个模拟时间单位，归一化改善约 0.0993%。HRL 未带来额外下降。单个合法案例的收益可能很小，完整性能仍需总体分析。这里的时间数值也不能直接解释为真实业务秒数。候选任务和节点决策保存在 candidate_schedules.jsonl，可供后续在同版本模拟器中逐任务核对时间线。

首先，验证集被重复使用。108 个场景同时用于模型选择和结果报告。基础 DAG 虽与训练集隔离，验证选择带来的偏差仍可能存在。应先冻结算法、配置和选模规则，再统一评估独立 test。

可训练分层策略和公平评估接口虽已实现，学习增益仍需单独证明。后续可通过冻结高层、固定低层启发式和控制更新幅度等对照，诊断退化原因。这些实验尚未完成。

求解成本和部署范围同样需要考虑。组合依次生成多个完整候选并重放，推理耗时高于 HEFT。离线确定性模拟器中的质量保证尚不能扩展到实际多机运行。本批实验未覆盖动态到达、节点故障、抢占和真实网络拥塞，也未证明能耗、成本或截止期得到联合优化。

工程归档仍有待补充的内容。双系统短训练与验证结果已保存，openEuler 具有完整验收状态；openKylin 还应补齐逐项测试和重载过程日志。本机绘图兼容问题、缺失的历史 smoke 配置及版本对应关系仍需处理。

本阶段已完成统一调度框架、三模型正式训练评测及双系统 CPU 短训练验证。合法候选组合在当前验证集上取得较低的平均归一化 makespan，收益来源和计算代价可以追溯。

第 5 章 实现难点说明

本项目主要有四项实现难点：统一不同策略的调度语义，保持分层训练采样与更新一致，保护多候选搜索中的合法结果，以及保存可核对的交付证据。本章介绍实际处理方法和验证范围。模型定义与算法流程见第 2 章，使用入口见第 3 章，完整性能结果见第 4 章

已有代码但缺少对应执行记录的内容，本章只介绍实现机制和测试判据。训练退化、历史版本差异和部分归档不足仍需说明。新增双系统记录用于更新平台完成情况。测试文件存在、测试实际通过和算法效果得到验证，分别对应不同层次的证据。

5.1 约束与调度语义一致性

离线 DAG 调度逐个决定任务放置位置，决策顺序与实际执行时间并不相同。若将“前驱已放置”理解为“前驱已在时间上完成”，可选任务集合就会改变。若屏蔽已安排任务的节点，也会丢失本可通过等待或插入空隙实现的合法方案。这些差异同时影响学习观测、启发式基线和最终评估，使策略缺少共同的比较基础。

项目将合法性判断和时间计算集中在环境与模拟器中。`ready_mask` 根据前驱是否已放置构造，`node_mask` 根据设备和容量约束筛选节点。`ScheduleSimulator` 接收动作后，再次检查可行性，计算依赖数据到达时间，并寻找节点上最早可用的连续空隙。策略负责提出选择，时间线只能通过环境更新。

表 5-1 容易混淆的语义及验证判据

具体困难	实际处理	对应验证
前驱已放置但尚未执行结束	允许后继进入待选集合，开始时刻仍受数据到达约束	后继不得早于前驱完成及传输结束
节点忙碌但满足资源要求	保留合法节点，由插入调度计算开始时刻	已有区间前的空隙可被新任务利用
同节点与跨节点通信混算	同节点通信为零，跨节点使用成对带宽和可选时延	比较两种放置下的数据就绪时刻
设备速度与通用速度混用	执行时间通过场景对应模型计算	GrapheonRL 设备速度进入独立候选审计
候选生成合法但重放输入变化	重放时重新检查 <code>ready_mask</code> 和 <code>node_mask</code>	对被篡改候选或不可行动作显式报错

`test_simulator.py` 提供了两个最小案例，用于解释调度语义。任务 a 在慢节点执行至时刻 10，依赖数据量为 20，跨节点带宽为 10、时延为 1。后继在另一节点的数据就绪时刻为 13，放在原节点则为 10。另一个案例先在节点上安排区间 [10,20]，再插入无前驱、时

长为 2 的任务，得到 [0,2]。这些断言直接检验模型行为，无需从大型实验均值间接推测正确性。

场景适配也影响比较的公平性。GrapheonRL 的执行时间需要使用任务设备对应的速度。若候选审计改用一般计算速度，候选自身评分与外部重放就可能不一致。项目通过执行模型接口连接候选生成和审计，并在 test_hybrid_demo.py 中设置设备速度审计及篡改拒绝测试。主要实现位于 scheduling/simulator.py、execution_model.py、communication_model.py、env/dag_env.py 和 demo 模块。

这些处理统一了当前模拟模型的约束。系统尚未模拟同节点多任务并发共享容量、真实链路争用、抢占或动态到达。容量可行只说明满足过滤条件，模拟器中的合法调度也仍需在真实多机执行中检验时间关系。

5.2 分层学习稳定性与贡献归因

高层任务选择改变低层面对的任务条件，低层分配又影响后续时间线和两层奖励。两层同时更新时，各层采集的状态分布会随另一层变化。分阶段训练是否解决了这一耦合问题，需要检查每阶段的动作来源、梯度计算和优化器更新，并结合最终验证表现判断。

项目用 DecisionPolicy 明确描述采样方式和可更新层。正式批次前 864 episodes 使用 high_only_eft，高层采样并更新，低层采用固定最小 EFT。后 1728 episodes 使用 joint，两层分别采样并更新。冻结同时约束梯度、模型模式和学习率调度器，避免仅跳过 optimizer.step 后仍发生其他状态变化。阶段配置也会拒绝“更新某层，但动作并非由该层 PPO 采样”等矛盾组合。

另一个关键点是采样概率与合法集合一致。PPO 将观测、动作、mask、旧对数概率和值估计一起存入轨迹，更新时复用原 mask。若在环境状态变化后重新计算合法集合，新旧概率比就不再对应同一动作分布。实现按张量形状分组处理不同任务规模，并在采样和更新前恢复所需的训练模式。

test_stage1_diagnostics.py 检查冻结模块参数不变、活动模块发生更新，以及采样动作与概率一致。test_ppo_mask.py 检查唯一合法动作的采样和可变任务数更新。test_ppo_mode.py 检查评估后是否恢复训练模式。这些测试用于排查训练流程错误，学习增益仍需通过正式验证结果分析。

表 5-2 正式批次的 HRL 验证轨迹

seed	step 0 与最佳点	后续验证及末轮结果
2026	step 0 为 0.999908；最佳 step 864 为 0.997358	step 1728 为 1.002619；step 2592 为 1.006308

seed	step 0 与最佳点	后续验证及末轮结果
2027	step 0 为 0.999908，且被选为最佳	step 864 为 1.011029；step 1728 为 1.007827；末轮为 1.029647
2028	step 0 为 0.999908，且被选为最佳	step 864 为 1.000515；step 1728 为 1.015656；末轮为 1.042573

表中数值为固定 validation 上的 mean_ratio，越小越好，来自 hrl_validation_history.csv。选模保留初始候选，避免后期较差权重因训练结束而覆盖最佳版本。这项措施保护的是已观测验证表现，尚不能证明模型收敛或在未见数据上泛化。验证集参与选模的限制与第 4 章一致。

目前可以确认，训练流程能够执行，冻结和选模机制明确，后期退化也已有记录。更新耦合、低层策略偏移或更新幅度是否为主要原因，仍需受控实验判断。后续可固定数据与种子，分别冻结高层、固定低层 EFT 或调整更新幅度，并同时保存初始、最佳和末轮结果。这些诊断尚未完成。

5.3 搜索计算开销与结果保护

搜索需要反复生成完整调度。若每次换节点或调整次序都从空状态重放，改动前相同的放置会被重复计算。若直接共享可变状态以减少重放，不同候选又可能相互影响。实现需要在复用计算的同时，保留每条候选独立、精确的时间线。

ScheduleSimulator.clone 复用静态场景信息，复制动态调度状态。搜索根据父候选和前缀深度保存快照，复用改动位置之前的前缀，再克隆快照并重新模拟后缀。切换父候选时，必须重建对应的前缀缓存，不能仅凭深度在不同调度间复用。正式配置已启用前缀缓存。代码中的结果缓存是另一项可选功能，本批次是否启用仍应以配置为准。

test_simulator.py 验证分支放置不会改变父状态。test_advanced_search.py 比较缓存开关下的候选结果，用模拟调用计数检查重复重放是否减少，并检查不同深度访问时的快照隔离。这些验证支持语义等价和重复计算减少，实际耗时降幅仍需单独测量。墙钟时间还受任务规模、候选数量、硬件和其他开销影响。

表 5-3 搜索处理与结果保护

风险	实际处理	保留的限制
估计代价好但真实完成时间差	估计只筛选候选，统一模拟器重放后评分	筛选预算仍可能遗漏更好候选
只看 DAG 路径忽略节点阻塞	实际关键链同时考虑数据到达及节点前序任务	有限关键链和短块不覆盖全部搜索空间

风险	实际处理	保留的限制
候选分支污染已有时间线	前缀快照克隆与动态状态隔离	需持续验证新移动方式的缓存等价性
失败候选被隐藏或旧计划被误用	失败报错并清空旧计划；最终逐动作检查	质量关系以全部候选成功完成为前提
搜索无限扩展影响交付	固定轮次、beam 宽度和块长度；无可扩展项时停止	固定预算不等于实时截止时间保证

beam 可以继续扩展暂时没有改善的分支，单轮无改善并非所有搜索的共同停止条件。关键块阶段则在没有移动候选，或本轮最优完成时间未改善超过 10^{-12} 时结束。整个搜索保留已有最优候选，也包含 HEFT。独立组合再比较 HEFT、搜索、HRL 的完整合法结果精确相等时按既定顺序选择，持平不计为学习贡献。

test_independent_hybrid.py 检查独立搜索是否不受 HRL 改变、精确持平时是否优先 HEFT，以及非法候选是否报错、旧计划是否清空。失败分支不能作为可用的低分候选。只有三路均完成并采用同一确定性评分时，组合完成时间才等于三者最小值。生成全部候选和最终重放的成本仍需计入。

正式批次三种子汇总的平均推理耗时为 HEFT 175.277 ms、纯搜索 4434.988 ms、组合 5296.759 ms，来自同一结果汇总文件。当前系统通过额外求解时间改善调度质量。这些数值未用于推导前缀缓存的提速比例，也未混入其他机器的历史计时。缓存的实际收益仍需在同环境、同候选预算和同输入下进行开关对照。

实现见 policies/portfolio.py 中的 _prefix_template 、 _beam_search 、 _realized_critical_chain 和关键块重放，以及 policies/hybrid.py 中的候选生成与重放检查。

5.4 平台与可复现交付

跨环境运行时，Python 和 PyTorch 版本、图形后端、CPU 线程、设备选择及路径编码都可能影响执行。产物写出后，流程也可能尚未全部结束。仅看到目录、截图或 best.pt，仍不足以确认训练、重载、验证和报告全部成功。

项目通过保存依赖版本、系统信息、源码提交和数据校验值，明确运行结果的适用环境。两个虚拟机配置均采用 CPU 单线程；openEuler 与 openKylin 分别记录自己的发行版和内核。历史容器配方及演示启动器的设置继续保留。验收时应区分容器、虚拟机和裸机，避免把某一环境的结果推广到其他部署形式。

验收还检查结果内容。verify_report 从逐场景文件核对 108 个唯一场景、54 个基础 DAG、合法性和有限正值，并重新计算均值。compare_replay 比较训练后评估结果与新进

程加载权重后的结果。`test_platform_acceptance.py` 用修改后的逐场景数据检查能否发现不一致，也检查异常是否写入 `failed`、旧证据目录是否受到覆盖保护。测试中的合成数据只用于验收逻辑检查。

表 5-4 平台运行与交付证据

对象	已有记录	当前结论边界
openEuler 虚拟机	188 项测试通过；六阶段及总状态 <code>passed</code> ；重载结果一致	对应 <code>x86_64</code> CPU 验收配置
openKylin 虚拟机	训练及 HRL/HEFT 验证结果齐备；README 记录 188 项测试与重载完成	还需补齐逐项测试及重载过程日志
双系统场景结果	各 108 场景、54 个基础 DAG；HRL <code>mean_ratio</code> 均为 0.999635；合法率 100%	使用同一组验证场景，不能计为 216 个独立样本
六场景演示	<code>complete</code> ；14 项演示测试、24 次时间线检查	覆盖六场景四方法，不代替全验证集或平台验收
本机绘图兼容	既有规划及第 4 章保留了崩溃问题	尚无最终闭环证据，不因容器配置 <code>Agg</code> 就宣称已解决
版本与材料归档	配置、权重和部分源码哈希可追溯	组合文件差异、来源许可及最终材料完整性仍需核对

双系统归档已支持 `x86_64` 虚拟机中的短训练和验证结论。`openEuler` 的新记录补齐了完整验收流程；`openKylin` 的数值结果可由逐场景文件复核，过程材料仍可补充。旧容器的 `running` 状态不作改写。本机绘图问题也应按原环境单独复现和验证，不能用虚拟机测试通过代替修复记录。

下一步应完善版本与交付对应关系：解释当前源码与正式实验冻结清单的差异，保留各平台的依赖、输入哈希和源码提交，补齐 `openKylin` 的测试及重载日志，并保存本机绘图问题的修复验证。算法、平台配置和文档的改动应分别记录，后续性能评估仍需采用统一预算与独立数据。

第 6 章 总结

本项目面向云边端异构资源上的离线 DAG 调度，已实现统一场景、约束处理、分层策略训练、策略接入和结果评估。任务选择与资源分配通过同一环境连接，执行和通信时间由统一模拟器计算。输出包括完整调度和可追溯记录。项目提供可独立运行的学习模块，以及独立搜索和候选验证选择机制，可分别评价框架完成度、学习表现和组合效果。

6.1 已完成工作与实验结论

项目已实现数据导入、资源配置替换、两级 LSTM/GAT 策略、掩码 PPO 和分阶段训练。HEFT 与学习比较方法共用调度接口。正式批次完成三个随机种子的训练和验证，保存了配置、权重、日志及逐场景指标。六场景离线演示支持方案比较、时间线与依赖查看和结果导出。新增 openEuler、openKylin 虚拟机记录补充了 CPU 短训练与验证证据，openEuler 的完整验收状态为 passed。

表 6-1 目标完成情况

目标	已有成果	仍需补足的范围
可配置且可插拔的调度框架	场景、资源、环境和策略接口分离；多策略统一评估	新配置与新策略仍需按相同协议验证
可训练的分层策略	三种子训练、阶段控制、验证选模及权重记录	后期退化仍存在，未证明稳定学习增益
改善当前模拟目标	独立搜索与候选校验组合取得较低验证 <code>mean_ratio</code>	搜索开销较大，独立测试尚待进行
可追溯的使用与交付	六场景演示、双系统 CPU 短训练验证、配置与证据索引	openKylin 过程日志、版本差异与来源材料仍需补齐

在固定 108 个验证场景、54 个基础 DAG 上，HRL 搜索方法三种子的 `mean_ratio` 为 0.918223 ± 0.000498 ，其中“ $\pm$ ”表示种子间样本标准差。相对 HEFT，平均归一化 `makespan` 降低约 8.18%，归档调度合法率为 100%。三个种子使用同一组验证场景，不能视为 324 个独立场景。该降幅描述的是归一化 `makespan`，有别于实际业务时延或每个任务执行时间的平均降幅。

HRL 在 seed 2026 的最佳检查点为 step 864，另两个种子选择 step 0，后期验证出现退化。现有证据支持统一框架可训练，HRL 是否能稳定显著提升搜索，以及增加训练轮次是否继续获益，仍需验证。

降低 `makespan` 需要额外求解时间。同一正式批次汇总的平均推理时间约为 HEFT 0.175 s、纯搜索 4.435 s、组合 5.297 s。组合耗时包括候选生成、校验、选择和重放，应

完整报告。候选最小值关系要求所有候选完整、合法，并采用同一确定性评分，尚不能据此保证全局最优或真实部署质量。

双系统验证中，openEuler 和 openKylin 各完成 2 个训练回合、400 次任务决策及 108 个场景评估。HRL 与 HEFT 的合法调度率均为 100%。openEuler 完整验收通过；openKylin 的测试和重载完成情况见归档说明。该结果补充了 CPU 虚拟机兼容性证据，不改变三种子正式性能实验的统计结论。

6.2 现有限制与后续工作

当前验证集参与了检查点选择和后续方案开发，冻结方案后的独立测试尚未完成。系统采用离线、不抢占的节点串行时间线，尚未验证动态到达、真实网络拥塞、节点故障或多机执行。能耗、费用和截止期也未纳入已完成的联合目标。双系统证据限于 x86_64 虚拟机 CPU 环境，裸机、ARM/鲲鹏与昇腾仍需另行验收。

后续应完善版本和交付证据。具体包括解释当前组合模块与正式实验冻结哈希的差异，补充源码来源、许可证及初赛决赛版本对照，补齐 openKylin 过程日志，并保存本机绘图问题的修复验证。报告、代码、权重和运行记录应能对应到明确版本。

学习方面，应先通过固定高层、固定低层 EFT 和控制更新幅度等对照诊断退化，再决定是否扩大训练。也可研究利用训练集内部搜索生成教师轨迹的监督方式，但需隔离验证数据，并单独报告计算预算和纯学习推理结果。

搜索方面，应在同环境、同场景和同候选预算下，比较缓存与候选裁剪的质量和耗时，检查减少重放能否带来实际运行收益。模型和搜索设置冻结后，再用未参与开发的独立数据评估泛化。接入真实系统时，还需新增资源观测、任务执行反馈和异常处理接口，并重新验证通信与执行时间假设。

本阶段已将可训练分层策略、统一约束模拟、独立候选改进和可复查评估连接为可用的调度框架，并完成双系统 CPU 短训练验证。后续重点是提高学习贡献与计算效率，开展独立验证，并完善版本归档和更多部署环境的运行证据。

参考

- [1] Arabnejad H, Barbosa J G. 2014. List Scheduling Algorithm for Heterogeneous Systems by an Optimistic Cost Table. *IEEE Transactions on Parallel and Distributed Systems*, 25(3): 682–694. DOI: 10.1109/TPDS.2013.57.
- [2] Hamilton W L, Ying R, Leskovec J. 2017. Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems*, 30. arXiv:1706.02216.
- [3] Henderson P, Islam R, Bachman P, Pineau J, Precup D, Meger D. 2018. Deep Reinforcement Learning That Matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). DOI: 10.1609/aaai.v32i1.11694.
- [4] Hochreiter S, Schmidhuber J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8): 1735–1780.
- [5] Kipf T N, Welling M. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*. arXiv:1609.02907.
- [6] Mao H, Schwarzkopf M, Venkatakrishnan S B, Meng Z, Alizadeh M. 2019. Learning Scheduling Algorithms for Data Processing Clusters. *Proceedings of the ACM Special Interest Group on Data Communication*: 270–288. DOI: 10.1145/3341302.3342080.
- [7] Schulman J, Moritz P, Levine S, Jordan M, Abbeel P. 2015. High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438.
- [8] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- [9] Topcuoglu H, Hariri S, Wu M. 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3): 260–274. DOI: 10.1109/71.993206.
- [10] Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y. 2018. Graph Attention Networks. *ICLR*. arXiv:1710.10903.
- [11] Williams R J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3–4): 229–256. DOI: 10.1007/BF00992696.
- [12] Yue Q, Tian L, Feng X, 等. 2026. CPN-HRL: a hierarchical deep reinforcement learning approach for priority-aware task scheduling in CPN enabled by cloud-edge-end environments. *Journal of King Saud University Computer and Information Sciences*, 38:290. DOI: 10.1007/s44443-026-00690-x.